Lab 15 — DoS Simulation and Detection

> **SY0-701 Objective 2.4** — Given a scenario, analyze indicators of malicious activity
>
> **Domain 2** — Threats, Vulnerabilities, and Mitigations (22%) · **Time:** ~30 min · **Course:** TGS-2023039181 CompTIA Security+ (SY0-701)

Goal: generate a controlled SYN flood against your own loopback, watch the connection table fill with half-open sockets, then mitigate it.

You'll build: a SYN-flood detection-and-mitigation exercise against a local nginx target.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; tools installed in step 1.

> **Legal note:** The flood targets only `127.0.0.1` inside your own playground. Directing flood traffic at any system you do not own is an offence under Singapore's Computer Misuse Act.

Background

- A **SYN flood** sends many TCP SYNs but never completes the handshake, leaving **half-open** connections that exhaust the backlog so legitimate clients are refused.
- DoS categories: **volumetric** (saturate bandwidth), **protocol/state** (exhaust connection tables — SYN flood), and **application-layer** (exhaust worker threads — Slowloris).
- **DDoS** sources the same attack from a botnet, so blocking one IP achieves nothing; **amplified/reflected** attacks spoof the victim's IP to a reflector (DNS, NTP, memcached) that returns oversized replies.
- Mitigations: **SYN cookies** (allocate no state until the ACK arrives), per-source **rate limiting**, and upstream scrubbing; DoS attacks the **availability** leg of the CIA triad.

Steps

1. Install the attacker and target

```
apt-get update -qq && apt-get install -y hping3 nginx iptables procps
service nginx start || systemctl start nginx
```

2. Baseline the connection states

Record the normal socket summary before any attack.

```
ss -s
ss -tan state syn-recv | wc -l
```

3. Launch a loopback-only SYN flood

`hping3` floods SYNs at the local web port for ten seconds, then stops.

```
timeout 10 hping3 -S -p 80 --flood 127.0.0.1 >/dev/null 2>&1 &
echo "flood running for 10s"
```

4. Watch the half-open connections grow

Sample the SYN-RECV count a few times while the flood runs.

```
for i in 1 2 3 4 5; do echo "syn-recv: $(ss -tan state syn-recv | wc -l)"; sleep 1; done
```

```
syn-recv: 0
syn-recv: 128      # backlog filling with half-open connections
syn-recv: 128
...
```

5. Enable SYN cookies

With SYN cookies on, the server encodes handshake state into the sequence number and allocates nothing until the ACK arrives.

```
sysctl -w net.ipv4.tcp_syncookies=1
```

6. Add an iptables rate limit

Cap new SYNs to the web port so a burst cannot fill the backlog.

```
iptables -I INPUT -p tcp --syn --dport 80 -m limit --limit 25/s --limit-burst 50 -j ACCEPT
iptables -A INPUT -p tcp --syn --dport 80 -j DROP
iptables -L INPUT -n --line-numbers | head
```

7. Re-run the flood and compare

With both mitigations active the half-open count stays bounded.

```
timeout 10 hping3 -S -p 80 --flood 127.0.0.1 >/dev/null 2>&1 &
for i in 1 2 3 4 5; do echo "syn-recv: $(ss -tan state syn-recv | wc -l)"; sleep 1; done
ss -s
```

Test it

Confirm SYN cookies are on and the rate-limit rule is in place.

```
sysctl net.ipv4.tcp_syncookies | grep -q ' = 1' && echo "SYN COOKIES ON"
iptables -S INPUT | grep -q 'limit 25/sec' && echo "RATE LIMIT RULE PRESENT"
echo "During the first flood SYN-RECV spiked; after mitigation it stays bounded."
```

Expected: SYN cookies read `1`, the rate-limit rule is listed, and the SYN-RECV count is visibly lower in step 7 than in step 4.

Troubleshooting

Symptom	Fix
`hping3: command not found`	Re-run step 1; on some images install `apt-get install -y hping3`.
SYN-RECV always 0	The loopback is very fast; raise visibility with a second `hping3 --faster` or watch `ss -s` totals instead.
iptables rule has no effect	Order matters — the DROP must come *after* the ACCEPT limit rule; re-list with `iptables -S INPUT`.

Challenge

Add an application-layer angle: throttle requests in nginx with `limit_req_zone`, then explain why on-premise rate limiting cannot stop a true volumetric DDoS and what upstream (ISP/CDN scrubbing) control is required instead.

Reflection

Objective 2.4 lists DDoS, amplified and reflected attacks. Which leg of the CIA triad does a DoS attack target, and how does that shape your incident-response priorities?

Exam tips

- A high ratio of SYN-RECV to ESTABLISHED connections is the classic SYN-flood indicator.
- SYN cookies defeat backlog exhaustion; rate limiting slows a single source but not a distributed botnet.
- Amplified/reflected DDoS spoofs the victim's IP so a reflector floods it with oversized replies.
- DoS attacks availability; confidentiality and integrity usually stay intact.

References

- CISA DDoS Guidance — https://www.cisa.gov/news-events/news/understanding-denial-service-attacks
- RFC 4987 TCP SYN Flooding — https://www.rfc-editor.org/rfc/rfc4987
- Cloudflare DDoS Learning Center — https://www.cloudflare.com/learning/ddos/
- BCP 38 Ingress Filtering — https://www.rfc-editor.org/info/bcp38
